ETSIIT
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Ingeniería Informática + ADE

Universidad de Granada (UGR)

Autor: Ismael Sallami Moreno

Asignatura: Ejercicios Tema 3 SCD: Paso de mensajes

Índice

1. Ejercicio 77 **3**
1.1. Enunciado 3
1.2. Solucion 3

1 Ejercicio 77

1.1. Enunciado

En un sistema distribuido, 6 procesos clientes necesitan sincronizarse de forma específica para realizar cierta tarea, de forma que dicha tarea sólo podrá ser realizada cuando tres procesos estén preparados para realizarla. Para ello, envían peticiones a un proceso controlador del recurso y esperan respuesta para poder realizar la tarea específica. El proceso controlador se encarga de asegurar la sincronización adecuada. Para ello, recibe y cuenta las peticiones que le llegan de los procesos, las dos primeras no son respondidas y producen la suspensión del proceso que envía la petición (debido a que se bloquea esperando respuesta) pero la tercera petición produce el desbloqueo de los tres procesos pendientes de respuesta. A continuación, una vez desbloqueados los tres procesos que han pedido (al recibir respuesta), inicializa la cuenta y procede cíclicamente de la misma forma sobre otras peticiones. El código de los procesos clientes aparece aquí abajo. Los clientes usan envío asíncrono seguro para realizar su petición, y esperan con una recepción síncrona antes de realizar la tarea:

```
process Cliente[ i : 0..5 ] ;
begin
  while true do begin
    send( peticion, Controlador );
    receive( permiso, Controlador );
    Realiza_tarea_grupal();
  end
end

process Controlador ;
begin
  while true do begin
    ...
  end
end
```

Describir en pseudocódigo el comportamiento del proceso controlador, utilizando una orden de espera selectiva que permita implementar la sincronización requerida entre los procesos. Es posible utilizar una sentencia del tipo select for i=... to ... para especificar diferentes ramas de una sentencia selectiva que comparten el mismo código dependiente del valor de un índice i.

1.2. Solucion

En un sistema distribuido, 6 procesos clientes necesitan sincronizarse de forma específica para realizar cierta tarea, de forma que dicha tarea sólo podrá ser realizada cuando tres procesos estén preparados para realizarla. Para ello, envían peticiones a un proceso controlador del recurso y esperan respuesta para poder realizar la tarea específica. El proceso controlador se encarga de asegurar la sincronización adecuada.

El controlador cuenta las peticiones que le llegan de los procesos:

- Las dos primeras peticiones no son respondidas, lo que provoca que los procesos que las envían queden bloqueados esperando una respuesta.

- La tercera petición produce el desbloqueo de los tres procesos pendientes, enviándoles una respuesta.

Una vez desbloqueados los tres procesos, el controlador inicializa la cuenta y procede cíclicamente de la misma forma para nuevas peticiones. Los clientes usan envío asíncrono seguro para realizar su petición y esperan con una recepción síncrona antes de realizar la tarea.

El código de los procesos clientes es el siguiente:

```
process Cliente[ i : 0..5 ] ;
begin
  while true do begin
    send( peticion, Controlador );   // Envía una petición al proceso controlador.
    receive( permiso, Controlador ); // Espera un permiso del controlador.
    Realiza_tarea_grupal();          // Realiza la tarea grupal una vez recibido el permiso.
  end
end
```

Listing 1: Código de los procesos clientes

El comportamiento del proceso controlador, que asegura la sincronización, se describe en pseudocódigo a continuación:

```
process Controlador ;
var
  contador : integer := 0;           // Cuenta las peticiones recibidas.
  buffer : array[0..2] of integer;   // Almacena los índices de los procesos en espera.

begin
  while true do begin
    select
      for i := 0 to 5                // Itera sobre las peticiones de los procesos clientes.
      when receive( peticion, Cliente[i] ) do
        buffer[contador] := i;       // Almacena el índice del proceso cliente en el buffer.
        contador := contador + 1;    // Incrementa el contador de peticiones.

        if contador = 3 then begin   // Si se han recibido tres peticiones ...
          for j := 0 to 2 do
            send( permiso, Cliente[buffer[j]] ); // Envía permiso a los tres procesos.
          contador := 0;             // Reinicia el contador.
        end
    end
  end
end
```

Listing 2: Pseudocódigo del proceso Controlador

Explicación del código del controlador:

- El proceso controlador mantiene un contador de peticiones y un buffer que almacena los índices de los clientes en espera.
- Por cada petición recibida, almacena el índice del cliente en el buffer y aumenta el contador.
- Cuando el contador alcanza tres, el controlador envía permisos a los tres procesos almacenados en el buffer y reinicia el contador.

Este enfoque asegura que los clientes se sincronizan correctamente antes de realizar la tarea grupal, tal como se requiere en el enunciado.